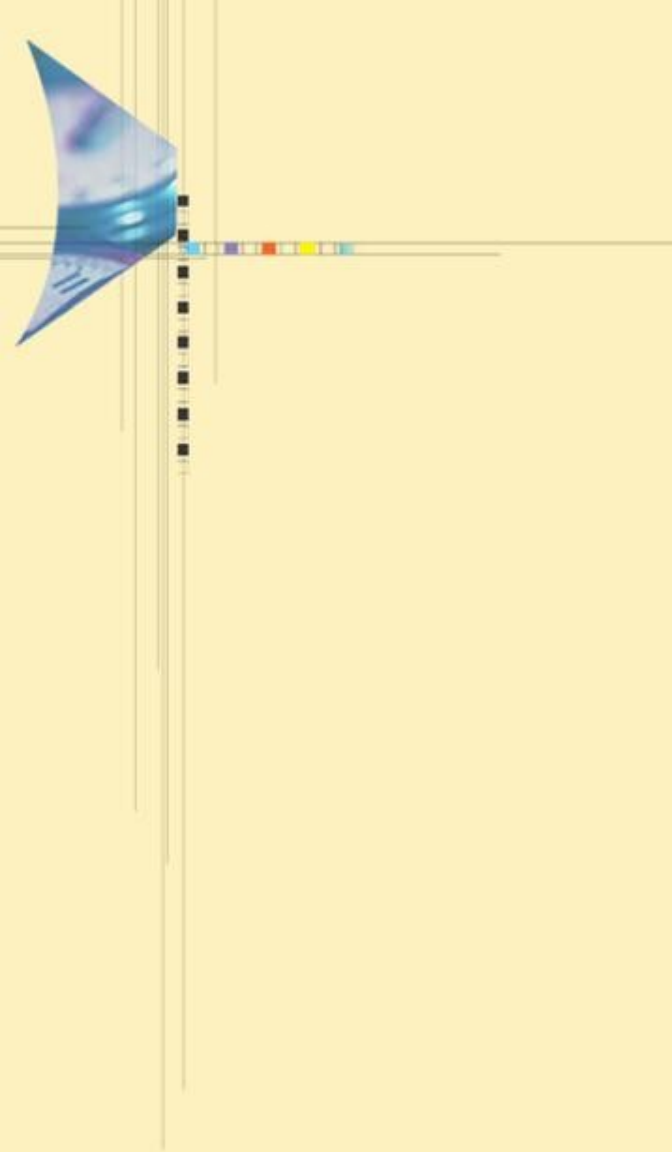




第 9 讲

分支限界法

本章内容

- 9.1 引言
 - 9.1.1 分支限界和回溯策略的异同点
 - 9.1.2 检索策略 (FIFO , LIFO , LC)
 - 9.1.3 简单问题例算法及分析 (不同检索策略的对比)
- 9.2 分支限界策略的算法设计思想及分析方法
 - 9.2.1 LC- 检索策略及优点
 - 9.2.2 LC- 分支限界策略的基本思想
 - 9.2.3 LC- 分支限界算法一般框架



本章内容

- 9.3. 分支限界策略的经典应用（可选讲部分）
 - 9.3.1 货郎担问题的算法描述、算法分析、程序实现
 - 9.3.2 0/1 背包问题的算法描述、算法分析、程序实现
 - 9.3.3 最大团集问题的算法描述、算法分析、程序实现
 - 9.3.4 圆排列问题的算法描述、算法分析、程序实现



9.1 引言

- 9.1.1 分支限界和回溯策略的异同点
- 9.1.2 检索策略（FIFO，LIFO，LC）
- 9.1.3 简单问题例算法及分析（不同检索策略的对比）

13.5 Branch and Bound(分支限界法)

- **分支 - 限界法**：在生成当前 E- 结点的全部儿子之后再生成其它活结点的儿子，且用限界函数帮助避免生成不包含答案结点子树的状态空间的检索方法。
- 检索策略分为：
 - **FIFO(first in first out) 检索**：类似于 BFS 的状态空间检索，它的活结点表采用一张先进先出表（队列）
 - **LIFO(last in first out) 检索**：类似于 D- 检索的状态空间检索，它的活结点表是一张后进先出表（栈）
 - **LC（Least Cost）- 检索（最小成本检索）**

分支界限法

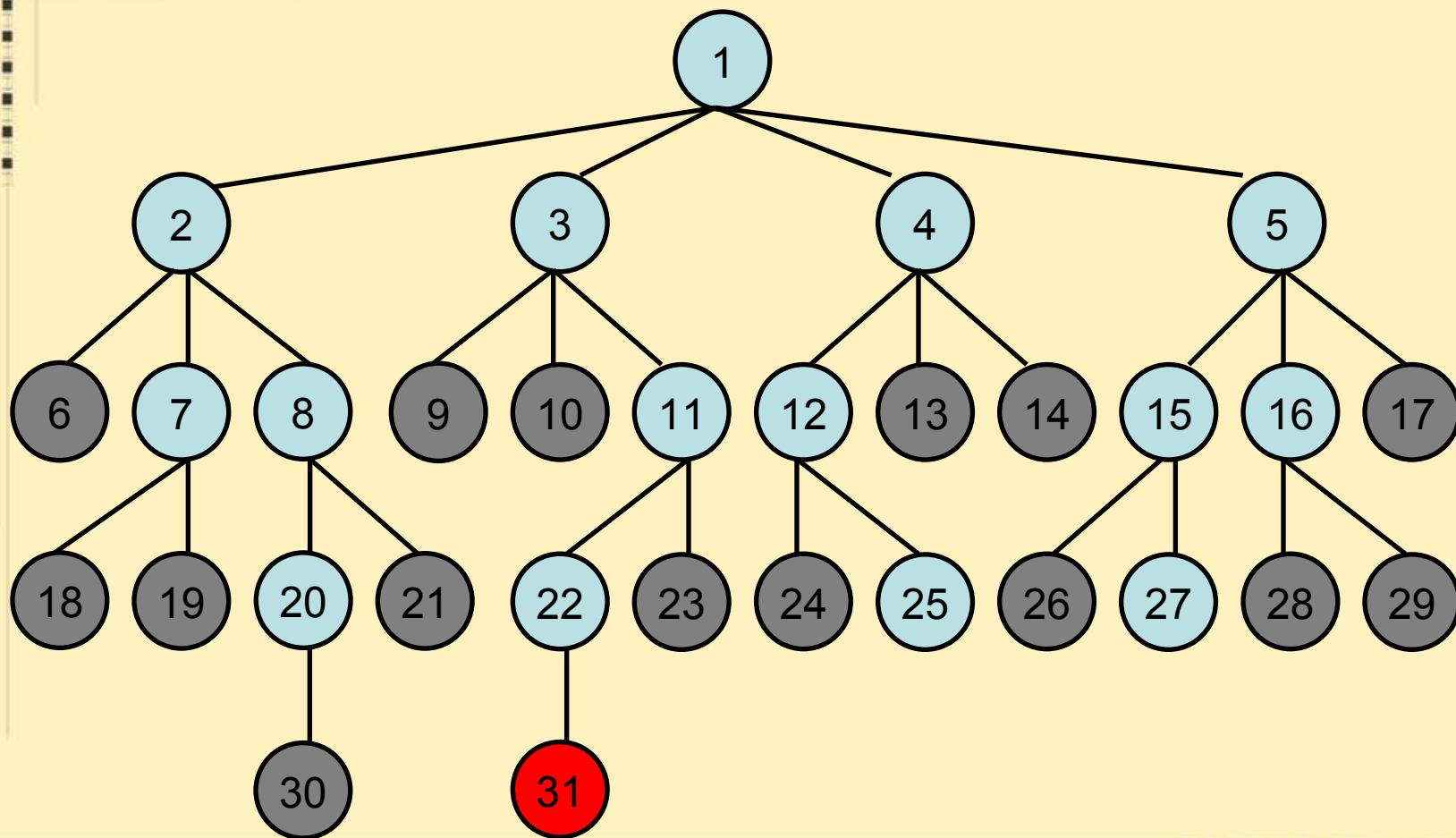
- 回溯法和分支限界法都是通过系统地搜索解空间树而求得问题解的搜索算法。

在系统地检查解空间的过程中，抛弃那些不可能导致合法解的候选解，从而使求解时间大大缩短。（与蛮干算法相区别）

- 回溯法以深度优先的方式搜索解空间，而分支限界法以广度优先或最小耗费（最大收益）的方式搜索解空间。



例子：4 - 皇后问题的 FIFO 分枝限界法



FIFO/LIFO 方法的缺陷

- **FIFO/LIFO- 分枝限界法缺点**：对下一个 E- 结点的选择相当死板，而且在某种意义上是盲目的。这种选择对于有可能快速检索到一个答案结点的结点**没有任何优先权**。
- 解决：对活结点使用一个“有智力的”**排序函数 $c(.)$** 来选取下一个 E- 结点，使有可能产生答案结点的活结点优先称为 E- 结点，从而可以加快到达一个答案结点的检索速度。
- 对任一结点 X，若要排序，必要进行计算，计算代价的度量标准：
 - (1) 在生成一个答案结点之前，子树 X 需要生成的结点数；
 - (2) 子树 X 中离 X 最近的那个答案结点到 X 的路径长度。

成本函数

- 结点成本函数 $c(.)$ 定义如下：
 - 如果 X 是答案结点，则 $c(X)$ 是由状态空间树的根结点到 X 的成本（即所用的代价，它可以是级数，计算复杂度等）；
 - 如果 X 不是答案结点且子树 X 不包含任何答案结点，则 $c(X)=\infty$ ；否则 $c(X)$ 等于子树 X 中具有最小成本的答案结点的成本。
- 问题：要得到结点成本函数 $c(.)$ 所用的计算工作量与解原问题具有相同的复杂度，这是因为计算一个结点的代价通常要检索包含一个答案结点的子树 X 才能确定，而这正是解决此问题所要做的工作。因此，要得到精确的成本函数一般是不现实的。
- 解决：定义新的成本估计函数： $\hat{c}(X)$ 。函数强使算法优先选择更靠近答案结点但又离根较近的结点。

LC- 检索

- 解决方法：
 - $\hat{g}(\cdot)$ ：结点成本的估计函数（由 X 到达一个答案结点所需做的附加工作的估计函数），但单纯使用 $\hat{g}(\cdot)$ 会导致算法偏向于做纵深检查。
 - 不只考虑结点 X 到一个答案结点的估计成本 $\hat{g}(\cdot)$ ，还考虑有根结点到结点 X 的成本 $h(X)$ 。
- 定义新的成本估计函数： $\hat{c}(X)=f(h(X)+\hat{g}(X))$ 。
- 其中， $\hat{g}(X)$ 是由 X 到达一个答案结点所需做的附加工作的估计函数； $h(X)$ 是由根结点到结点 X 的成本。 $f(\cdot)$ 是一个非降函数。f 函数强使算法优先选择更靠近答案结点但有离根较近的结点
- LC- 检索：选取下一个 E- 结点的检索策略总是选取 $\hat{c}(\cdot)$ 值最小的活结点作为下一个 E- 结点。称为最小成本检索，简称为 LC- 检索。
- LC 分枝 - 限界检索：伴有限界函数的 LC- 检索称为 LC 分枝 - 限界检索。



9.2 分支限界策略的算法设计思想及分析方法

- 9.2.1 LC- 检索策略及优点
- 9.2.2 LC- 分支限界策略的基本思想
- 9.2.3 LC- 分支限界算法一般框架



15- 谜问题 (例子)

- 问题描述：在一个分成 4×4 的方形棋盘上放有 15 块编号为 1, 2, ..., 15 的牌，给定这些牌的初始排列（例图 1），要求通过一系列合法的移动（只有相邻于空格的牌移动到空格才是合法的移动）将初始排列转换目标排列（如图 2）。

1	3	4	15
2		5	12
7	6	11	14
8	9	10	13

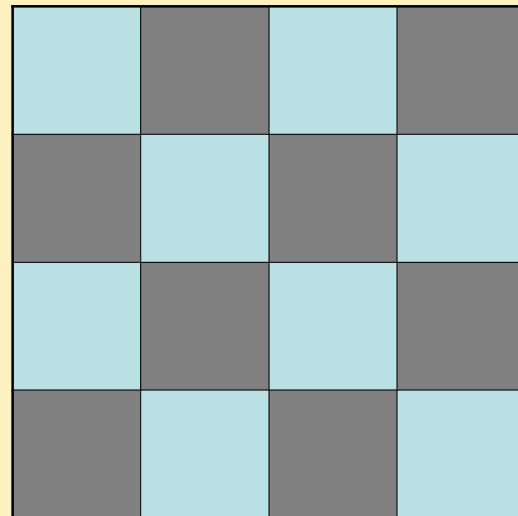
图 1

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

图 2

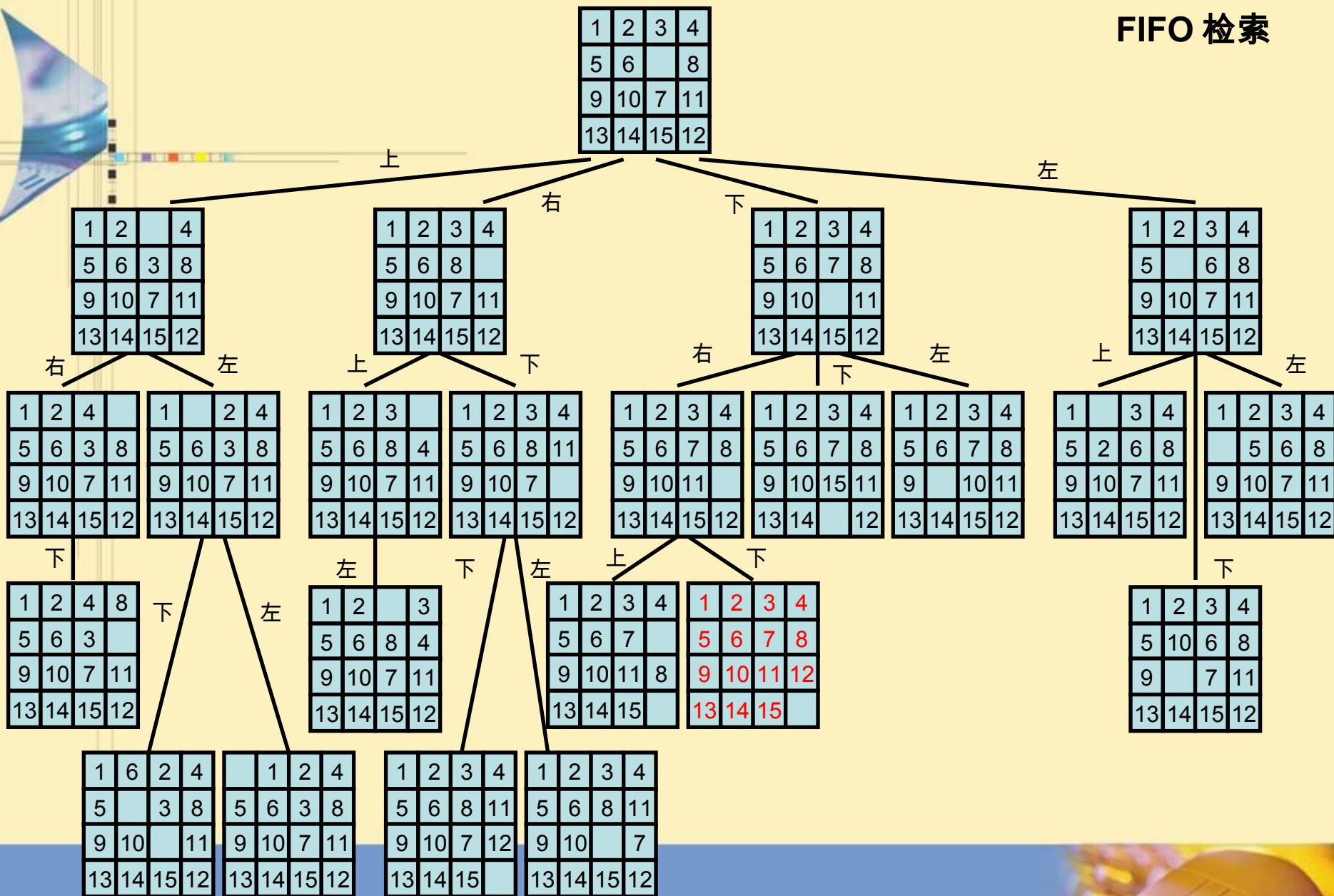
15- 谜问题 (例子)

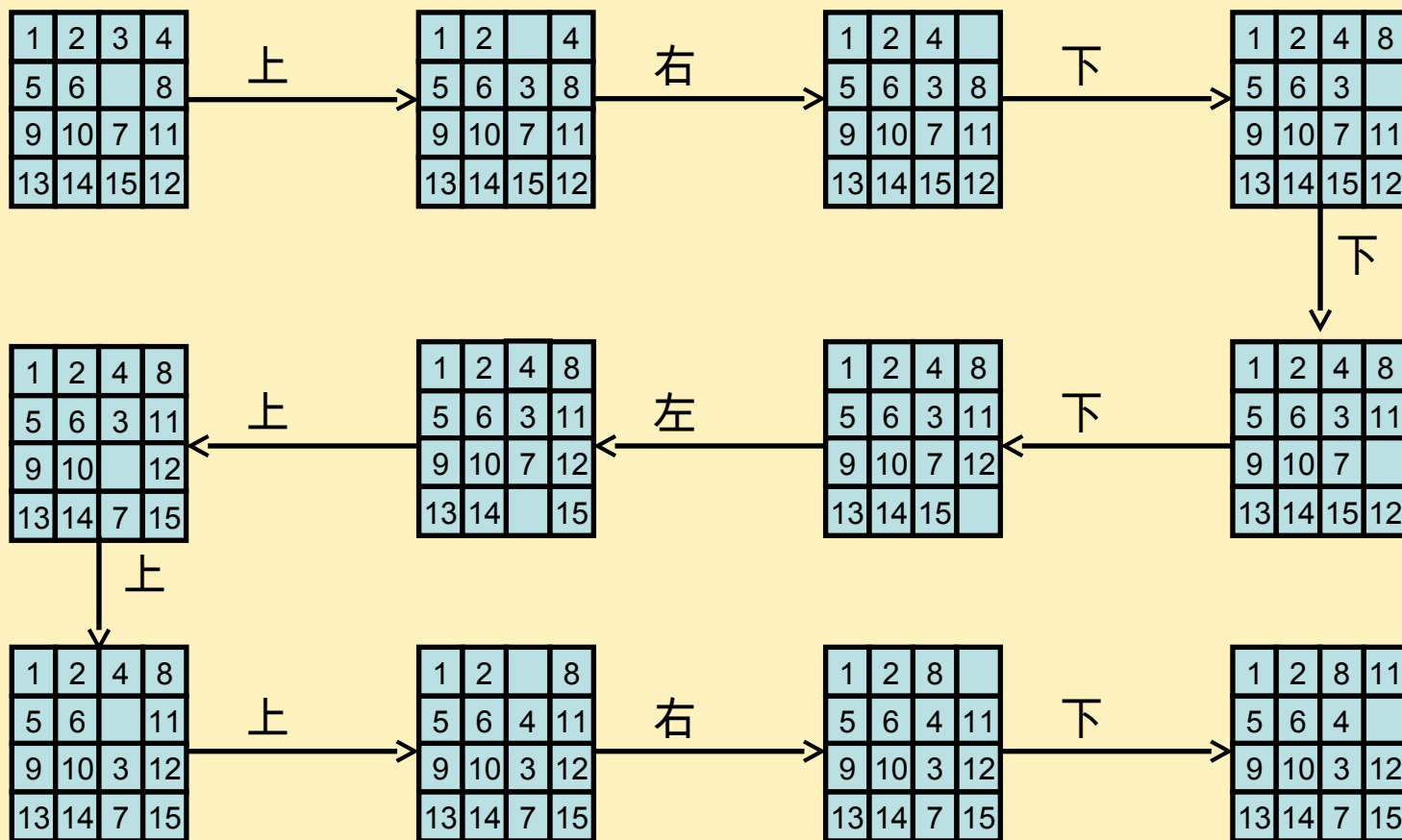
- 目标状态是否可由初始状态到达 (即目标状态是否在初始状态的状态空间中) 的判定:
 - 给棋盘的方格编上 $1 \sim 16$ 的编号, 位置 i 是目标排列中放 i 号牌的方格位置, 位置 16 是空格的位置;
 - 设 $POSITION(i)$ 表示编号为 i 的牌在初始状态下的位置号, $1 \leq i < 16, POSITION(16)$ 表示空格的位置;
 - 设 $LESS(i)$ 是使牌 j 小于牌 i 且 $POSITION(j) > POSITION(i)$ 的数目;
 - 在初始状态下, 如果空格在右图中阴影位置中的某一格处, 则令 $X=1$, 否则令 $X=0$ 。



定理：当且仅当 $LESS(1)+LESS(2)+\dots+LESS(16)+X$ 是偶数时，目标状态可由初始状态到达。

FIFO 检索

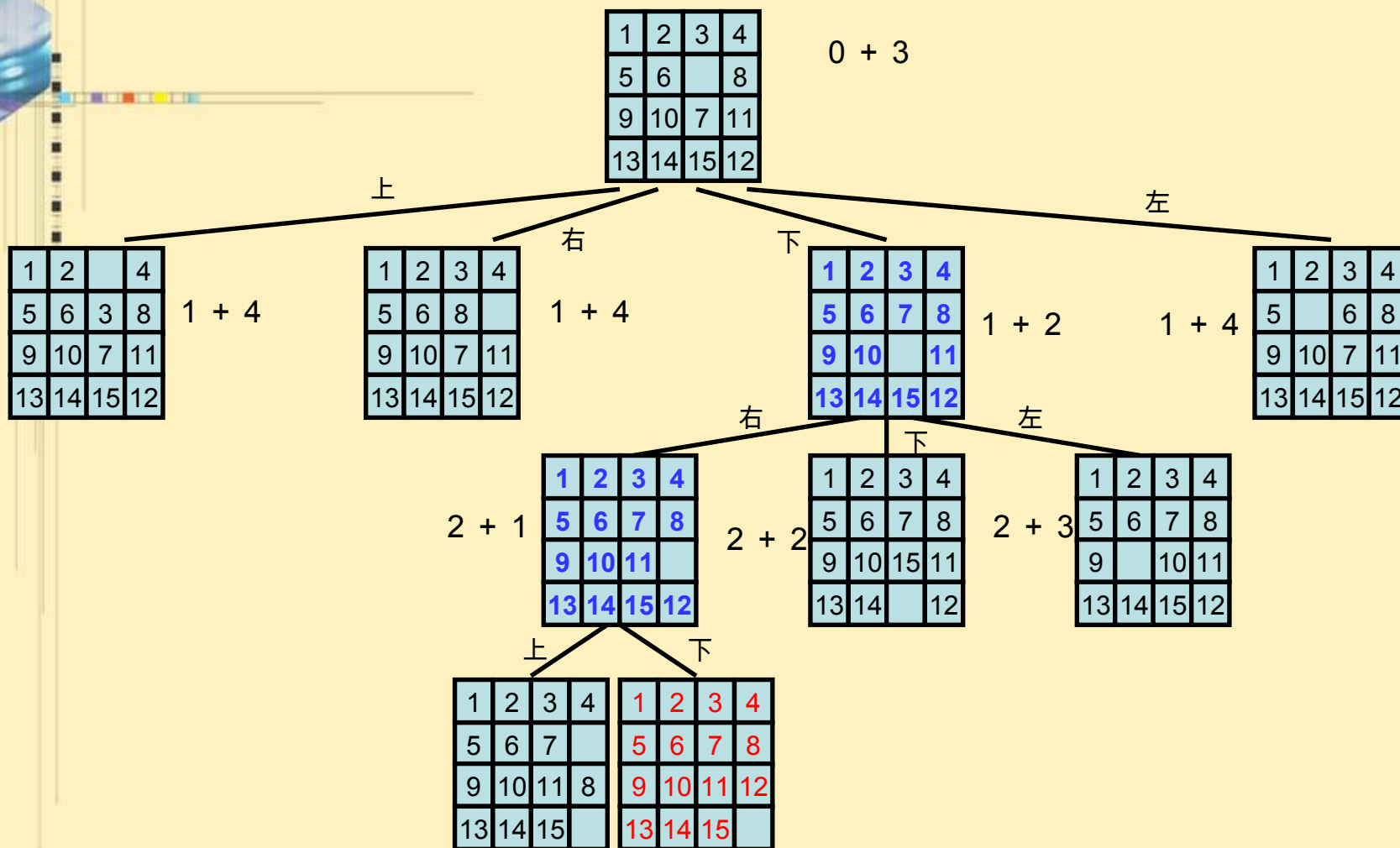




15- 谜问题的 LC- 检索

- 成本函数 $c(.)$: 从根出发到最近目标结点路径上的每个结点赋予这条路径的长度作为它们的成本。
- 估计成本函数 $\hat{c}(.)$: $\hat{c}(X)=h(X)+\hat{g}(X)$,
 - $h(X)$ 是由根到结点 X 的路径的长度;
 - $\hat{g}(X)$ 是对以 X 为根的子树中由 X 到目标状态的一条路径长度的估计。它至少应是能把状态 X 转换成目标状态所需的最小移动数。
$$\hat{g}(X)= \text{不在其目标位置的非空白牌数目}$$
- 可以看出 $\hat{c}(X)$ 是 $c(X)$ 的下界。

LC- 检索

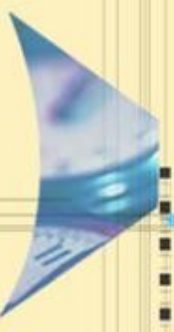


LC- 检索的一般过程

- LC- 检索与 FIFO, LIFO 的 **区别** 在于得到下一个 E- 结点的规则不同.
- $c(.)$: 状态空间树 T 中结点的成本函数; $c(X)$ 是其根为 X 的子树中任一答案结点的最小成本; $c(T)$ 是 T 中最小成本的答案结点的成本;
- 通常不可能找到如上定义的易于计算的 $c(.)$, 故用一个对 $c(.)$ 估值的 **启发式函数** $\hat{c}(.)$ 代替.
- $\hat{c}(.)$ 一般易于计算且有如下性质: 如果 X 是一个答案结点或是一个叶子结点, 则有 $c(X) = \hat{c}(X)$

找最小成本答案结点的分枝 - 限界算法

- 使用 $\hat{c}(X) \leq c(X)$ 的成本估计函数 $\hat{c}(\cdot)$ 来给出由任一结点 X 得出的解的下界，使算法具有一定的智能，减少盲目性；
- 定义 U 是最小成本解的成本上界，则：
 - (1) 具有 $\hat{c}(X) > U$ 的活结点 X 可以被杀死，这是因为由 X 可以到达的答案结点 X 有 $c(X) \hat{c}(X) > U$ ；
 - (2) 如果已经到达一个具有成本 U 的答案结点，则 $\hat{c}(X) \leq U$ 的所有活结点都可以被杀死
 - (3) U 的初始值可以用某种启发式方法得到，也可以置成 ∞



TSP Problem (旅行商问题)

? 问题描述

- 给出一个城市的集合和一个定义在每一对城市间的耗费函数，找出耗费最小的旅行。在这里，一个旅行是一个闭合的路径，它访问每个城市恰好一次，也就是说，是一个简单回路。耗费函数可以是距离、旅行时间、飞机票价等。
- An instance of the TRAVELING SALESMAN is given by its cost matrix whose entries are assumed to be nonnegative (非负的) .

估计成本函数

- 我们给每个部分解 $(x_1, x_2, \dots, x_j)$ 一个相关联的下界 y ，它解释如下，任何完整的巡回旅行，它按这种次序访问了城市 $x_1, x_2, \dots, x_j$ ，它的耗费必须至少是 y 。
- Each complete tour must contain exactly one edge and its associated cost from each row and each column of the cost matrix.
- If a constant r is subtracted from every entry in any row or column of the cost matrix A , the cost of any tour under the new matrix is exactly r less than the cost of the same tour under A .
 - This motivates the idea of reducing the cost matrix so that each row or column contains at least one entry that is equal to 0. We will refer to such a matrix as **the reduction of the original matrix**.
 - P225



例子

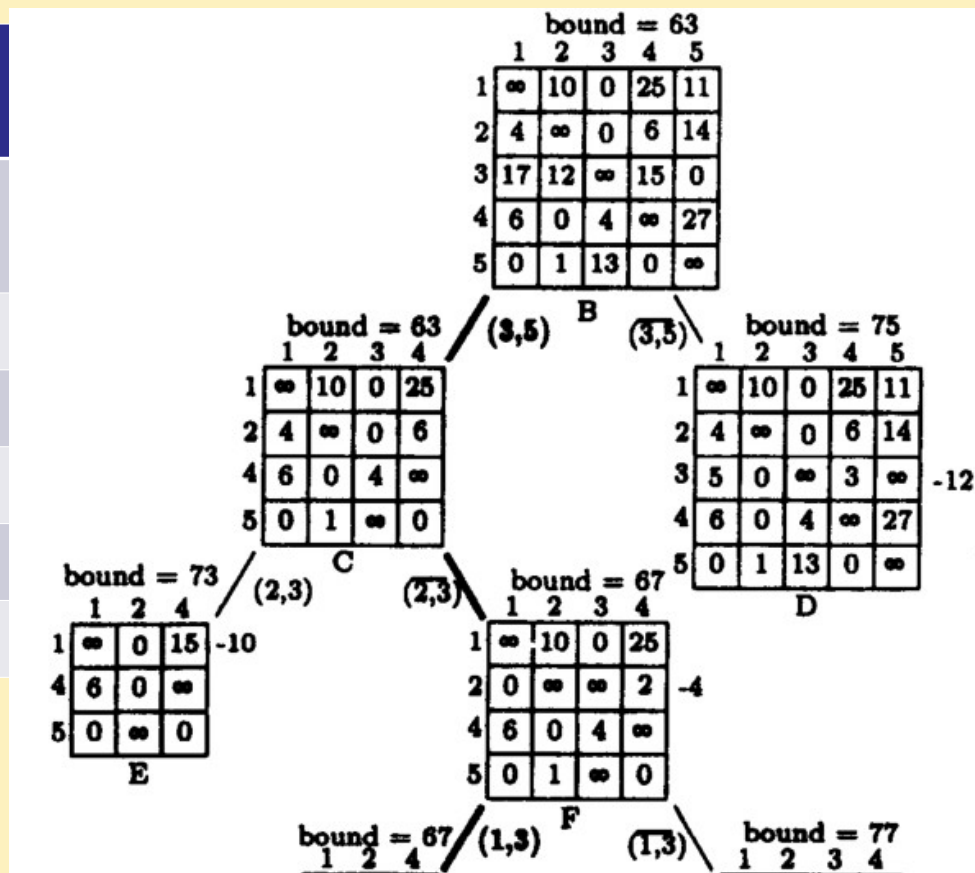
- 旅行商问题的实例矩阵和它的归约矩阵

∞	17	7	35	18	$\longrightarrow$	∞	10	0	28	11	-7
9	∞	5	14	19	$\longrightarrow$	4	∞	0	0	14	-5
29	24	∞	30	12	$\longrightarrow$	17	12	∞	18	0	-12
27	21	25	∞	48	$\longrightarrow$	6	0	4	∞	27	-21
15	16	28	18	∞	$\longrightarrow$	0	1	13	0	∞	-15

-3

$$y=7+5+12+21+15+3=63$$

$\text{Cost}(i,j)$ =0	不含 (i,j) 时, 代价增量
(1,3)	$\text{Cost}(1,2)+\text{Cost}(2,3)=10+0=10$
(2,3)	$\text{Cost}(2,1)+\text{Cost}(1,3)=4+0=4$
(3,5)	$\text{Cost}(3,2)+\text{Cost}(1,5)=12+11=23$
(4,2)	$\text{Cost}(4,3)+\text{Cost}(5,2)=4+1=5$
(5,1)	$\text{Cost}(5,4)+\text{Cost}(2,1)=0+4=4$
(5,4)	$\text{Cost}(5,1)+\text{Cost}(2,4)=0+6=6$



估计成本的计算

- 设 A 是结点 R 的归约成本矩阵， S 是 R 的儿子且 $\langle R, S \rangle$ 对应周游路线中的边 $\langle i, j \rangle$ 。
- 在 S 是非叶结点的情况下， S 的规约成本矩阵按以下方法获得：
 1. 为保证这条周游路线采用边 $\langle i, j \rangle$ 而不采用其它由 i 出发或者进入 j 的边，将 A 中 i 行和 j 列的元素置为 ∞ ；
 2. 为防止使用边 $\langle j, i \rangle$ ，将 A 中 j 行 i 列的元素置为 ∞ ；
 3. 对于那些不全为 ∞ 的行和列施行归约规则得到 S 的归约成本矩阵，令其为 B ，矩阵约数为 r ；
 4. S 的估计成本值可计算为：

$$\hat{c}(S) = \hat{c}(R) + A(i, j) + r$$

左右儿子的估计成本计算

- 设 A 是结点 R 的归约成本矩阵， S_L 是 R 的左儿子且对应包含边 $\langle i, j \rangle$ ， S_R 是 R 的右儿子且对应不包含边 $\langle i, j \rangle$ 。 S_L 的归约成本矩阵 B_L ，矩阵约数 r_L ； S_R 的归约成本矩阵 B_R 和矩阵约数 r_R 。
- 由于 S_R 代表不包含边 $\langle i, j \rangle$ 的周游，因此应将 R 的归约成本矩阵 A 中的元素 $A(i, j)$ 置成 ∞ 后，再归约此矩阵不全为 ∞ 的行和列（实际上只需重新归约第 i 行和第 j 列），即得 S_R 的归约成本矩阵 B_R 和矩阵约数

$$r_R = \min_{k \neq j} \{A(i, k)\} + \min_{k \neq i} \{A(k, j)\}$$

- 计算 $\hat{c}(S_R)$ 时，由于不包含边 $A(i, j)$ ，因此不必加入；
- 因此， R 的左、右儿子的估计成本值为：
 - $\hat{c}(S_L) = \hat{c}(R) + A(i, j) + r_L$
 - $\hat{c}(S_R) = \hat{c}(R) + r_R$

边的取舍次序

- 如果选取边 $\langle i, j \rangle$ ，则这条边将解空间分成两个子集合，根的左子树表示包含边 $\langle i, j \rangle$ 的所有周游，根的右子树表示不含边 $\langle i, j \rangle$ 的所有周游；
- 如果左子树中包含一条最小成本周游路线，则只需再选取 $n-1$ 条边；如果所有的最小成本周游路线都在右子树中，则还需要对边做 n 次选择；
- 在左子树中找最优解比在右子树中找最优解容易；
- 希望选取一条最有可能在最小成本周游路线中的边 $\langle i, j \rangle$ 作为分割边。
- 选择边的规则：
 - 选取一条使其右子树具有最大 $\hat{c}$ 值的边，OR，
 - 选取使左、右子树 $\hat{c}$ 值相差最大的边。。。

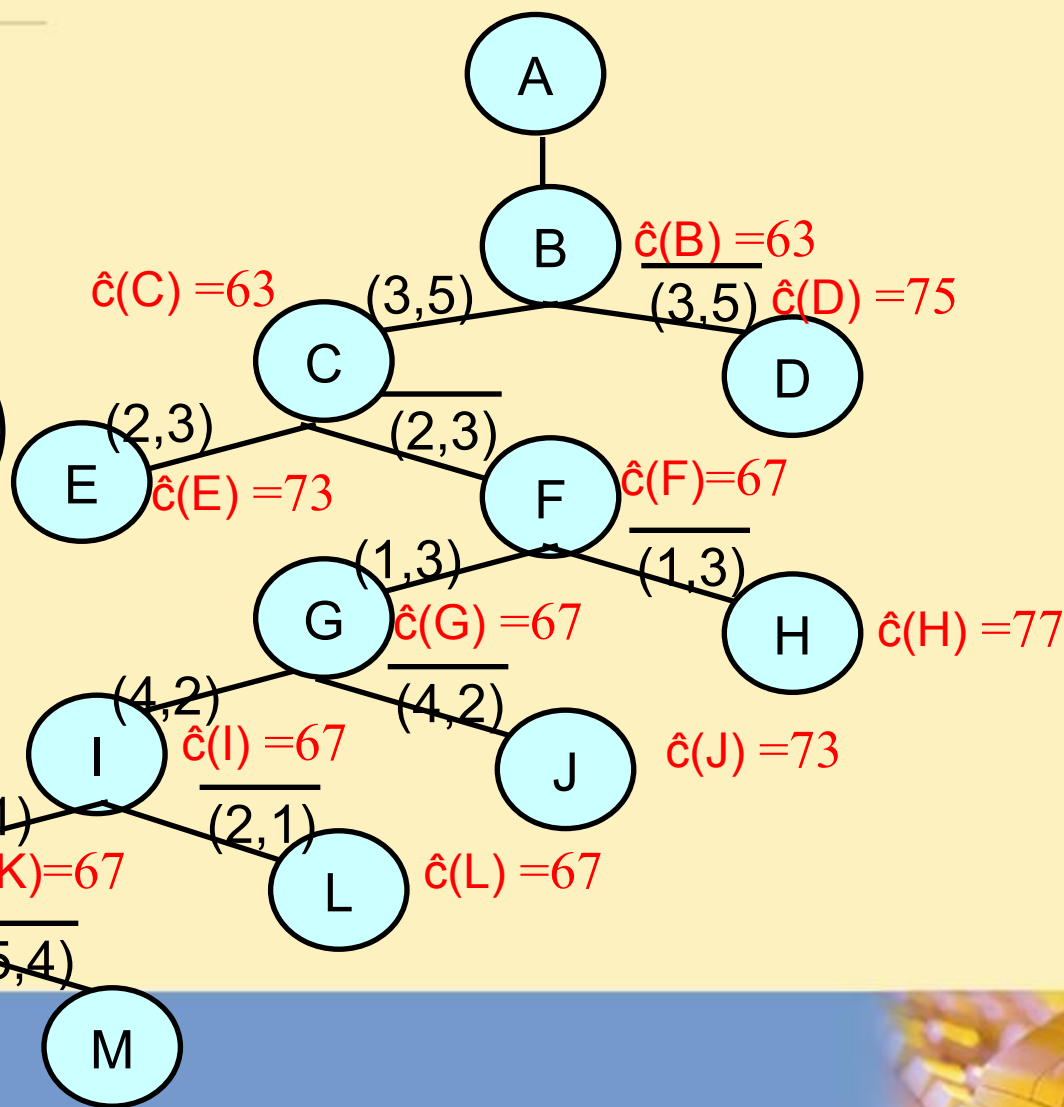
选取一条使其右子树具有最大 $\hat{c}$ 值的边

- 如果选取的边 $\langle i, j \rangle$ 在 R 的归约矩阵 A 中对应的元素 $A(i, j)$ 不为 0，则必有 $r_R = 0$ ，即 $\hat{c}(S_R) = \hat{c}(R)$ ；显然不能使 R 的右子树的 $\hat{c}$ 值最大；
- 因此，为了使 R 的右子树的 $\hat{c}$ 值最大，应从 R 的归约成本矩阵元素为 0 的对应边中选取有最大 r_R 值的边。



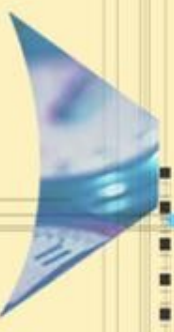
An example: the traveling salesman problem

∞	2	7	35	1
9	∞	5	14	19
4	24	∞	30	12
27	21	5	∞	3
15	16	28	18	∞



SOLUTION!





9.3. 分支限界策略的经典应用（可选讲部分）

- 9.3.1 货郎担问题的算法描述、算法分析、程序实现
- 9.3.2 0/1 背包问题的算法描述、算法分析、程序实现
- 9.3.3 最大团集问题的算法描述、算法分析、程序实现
- 9.3.4 圆排列问题的算法描述、算法分析、程序实现

9.3.1 货郎担问题

1. 问题描述

某售货员要到若干城市去推销商品，已知各城市之间的路程（或旅费）。他要选定一条从驻地出发，经过每个城市一次，最后回到驻地的路线，使总的路程（或总旅费）最小。

路线是一个带权图。图中各边的费用（权）为正数。图的一条周游路线是包括 V 中的每个顶点在内的一条回路。周游路线的费用是这条路线上所有边的费用之和。

旅行售货员问题的解空间可以组织成一棵树，从树的根结点到任一叶结点的路径定义了图的一条周游路线。旅行售货员问题要在图 G 中找出费用最小的周游路线。

9.3.1 货郎担问题

2. 算法描述

算法开始时创建一个最小堆，用于表示活结点优先队列。堆中每个结点的子树费用的下界 $lcost$ 值是优先队列的优先级。接着算法计算出图中每个顶点的最小费用出边并用 $minout$ 记录。如果所给的有向图中某个顶点没有出边，则该图不可能有回路，算法即告结束。如果每个顶点都有出边，则根据计算出的 $minout$ 作算法初始化。

算法的 $while$ 循环体完成对排列树内部结点的扩展。对于当前扩展结点，算法分 2 种情况进行处理：



9.3.1 货郎担问题

2. 算法描述

1、首先考虑 $s=n-2$ 的情形，此时当前扩展结点是排列树中某个叶结点的父结点。如果该叶结点相应一条可行回路且费用小于当前最小费用，则将该叶结点插入到优先队列中，否则舍去该叶结点。

2、当 $s < n-2$ 时，算法依次产生当前扩展结点的所有儿子结点。由于当前扩展结点所相应的路径是 $x[0:s]$ ，其可行儿子结点是从剩余顶点 $x[s+1:n-1]$ 中选取的顶点 $x[i]$ ，且 $(x[s], x[i])$ 是所给有向图 G 中的一条边。对于当前扩展结点的每一个可行儿子结点，计算出其前缀 $(x[0:s], x[i])$ 的费用 cc 和相应的下界 $lcost$ 。当 $lcost < bestc$ 时，将这个可行儿子结点插入到活结点优先队列中。

9.3.1 货郎担问题

2. 算法描述

算法中 while 循环的终止条件是排列树的一个叶结点成为当前扩展结点。当 $s=n-1$ 时，已找到的回路前缀是 $x[0:n-1]$ ，它已包含图 G 的所有 n 个顶点。因此，当 $s=n-1$ 时，相应的扩展结点表示一个叶结点。此时该叶结点所相应的回路的费用等于 cc 和 $lcost$ 的值。剩余的活结点的 $lcost$ 值不小于已找到的回路的费用。它们都不可能导致费用更小的回路。因此已找到的叶结点所相应的回路是一个最小费用旅行售货员回路，算法可以结束。

算法结束时返回找到的最小费用，相应的最优解由数组 v 给出。



9.3.2 0/1 背包问题

【问题描述】 有 n 个重量分别为 $\{w_1, w_2, \dots, w_n\}$ 的物品，它们的价值分别为 $\{v_1, v_2, \dots, v_n\}$ ，给定一个容量为 W 的背包。

设计从这些物品中选取一部分物品放入该背包的方案，每个物品要么选中要么不选中，要求选中的物品不仅能够放到背包中，而且重量和为 W 具有最大的价值。





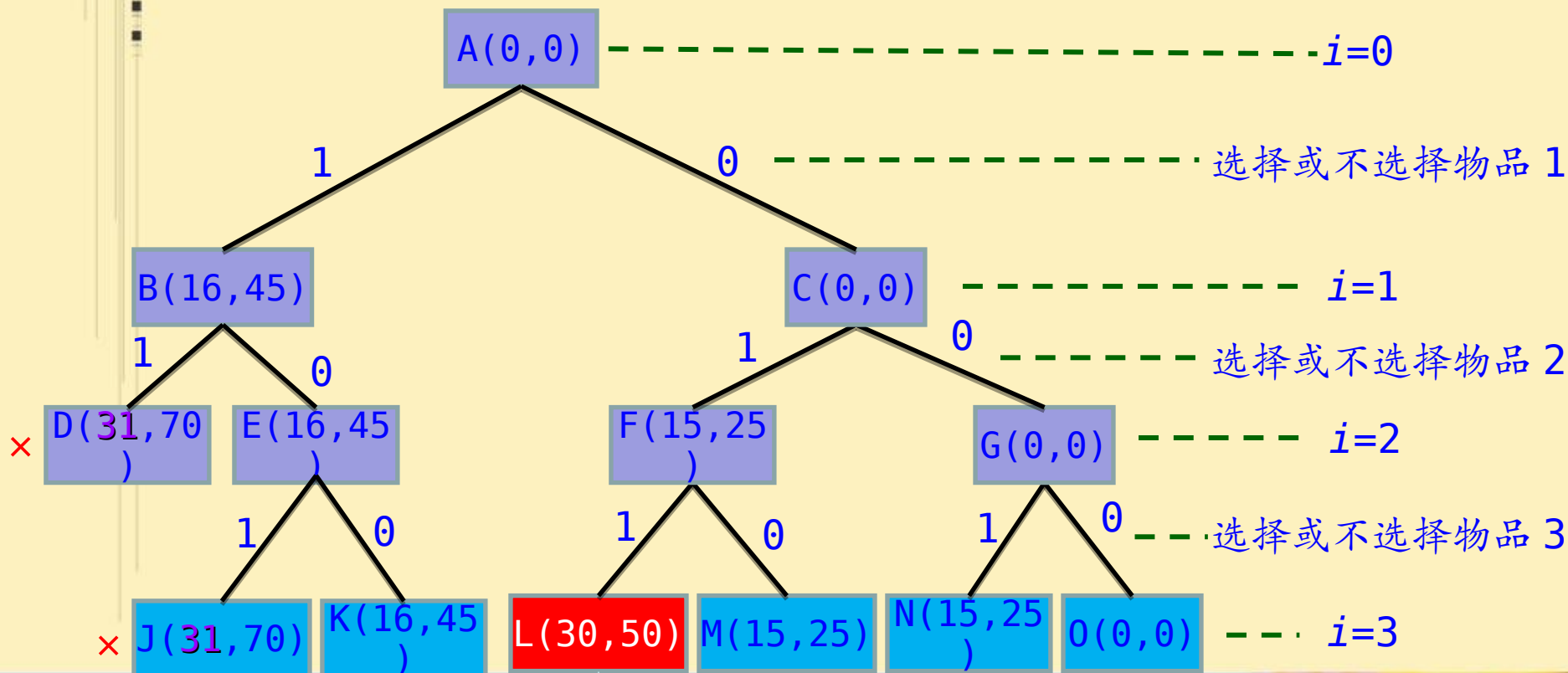
假设一个 0/1 背包问题是, $n=3$, 重量为 $w= (16, 15, 15)$, 价值为 $v= (45, 25, 25)$, 背包限重为 $W=30$, 解向量为 $x= (x_1, x_2, x_3)$ 。

编号	1	2	3
重量	16	15	15
价值	45	25	25

6.2.1 采用队列式分枝限界法求解

首先不考虑限界问题，用 **FIFO** 表示队列（实际上对应层次遍历）。初始时， $\text{FIFO}=[]$ 。

编号	1	2	3
重量	16	15	15
价值	45	25	25



可行解

得到最终解：(0, 1, 1)

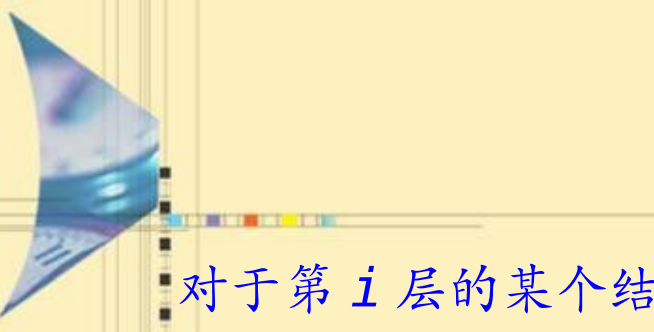
采用 STL 的 `queue<NodeType>` 容器 `qu` 作为队列，队列中的结点类型声明如下：

```
struct NodeType           // 队列中的结点类型
{   int no;               // 结点编号，从 1 开始
    int i;                // 当前结点在搜索空间中的层次
    int w;                // 当前结点的总重量
    int v;                // 当前结点的总价值
    int x[MAXN];          // 当前结点包含的解向量
    double ub;            // 上界
};
```



现在设计限界函数，为了简便，设根结点为第 0 层，然后各层依次递增，显然 $i=n$ 时表示是叶子结点层。

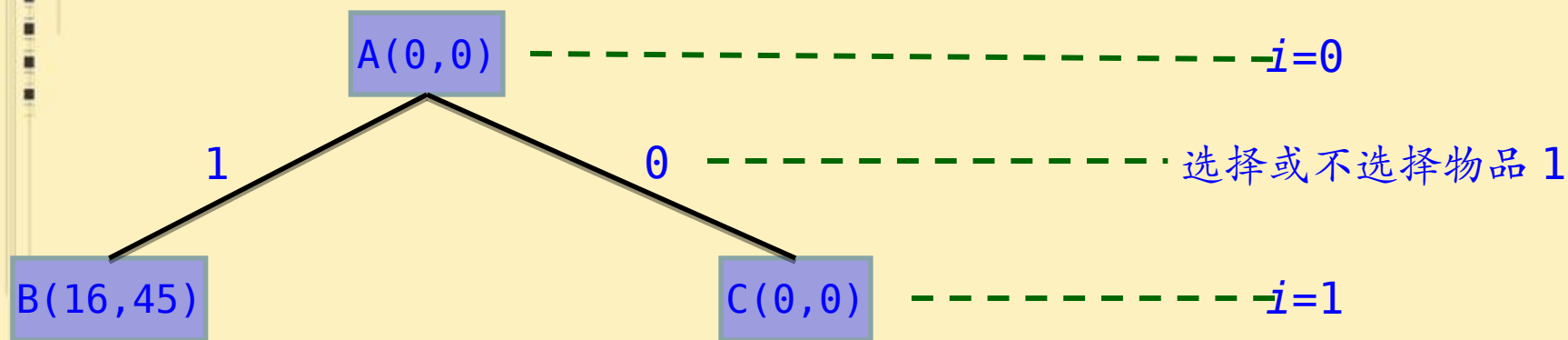
由于该问题是求装入背包的最大价值，属求最大值问题，采用上界设计方式。



对于第 i 层的某个结点 e ，用 $e.w$ 表示结点 e 时已装入的总重量，用 $e.v$ 表示已装入的总价值：

- 如果所有剩余的物品都能装入背包，那么价值的上界
$$e.ub = e.v + (v[i+1] + \dots + v[n])$$
- 如果所有剩余的物品不能全部装入背包，那么价值的上界
$$e.ub = e.v + (v[i+1] + \dots + v[k]) + (\text{物品 } k+1 \text{ 装入的部分重量}) \times \text{物品 } k+1 \text{ 的单位价值}$$

编号	1	2	3
重量	16	15	15
价值	45	25	25



根结点 A 的层次 $i=0$, $w=0$, $v=0$:

$$ub=0 + 45 + (30-16) \times 25/15 = 68 \quad (\text{采用取整运算})$$

$w=0$

$w[1]=16 < 30$
可选物品 1
 $v[1]=45$

可选物品 2 的
一部分, 即
 $30-16$, 对应的
价值

求结点 e 的上界 $e.ub$ 的算法如下:

```
void bound(NodeType &e)
{  int i=e.i+1;
   int sumw=e.w;
   double sumv=e.v;
   while ((sumw+w[i]<=W) && i<=n)
   {  sumw+=w[i];
      sumv+=v[i];
      i++;
   }

   if (i<=n)
       e.ub=sumv+(W-sumw)*v[i]/w[i];
   else
       e.ub=sumv;
}
```

// 计算分枝结点 e 的上界
// 考虑结点 e 的余下物品
// 求已装入的总重量
// 求已装入的总价值

// 计算背包已装入载重
// 计算背包已装入价值

// 余下物品只能部分装入

// 余下物品全部

可以装入

$e.ub = \text{sumv};$

// 问题表示

```
int n=3,W=30;
```

```
int w[]={0,16,15,15};
```

```
int v[]={0,45,25,25};
```

// 求解结果表示

```
int maxv=-9999;
```

```
int bestx[MAXN];
```

```
int total=1;
```

```
struct NodeType
```

```
{ int no;
```

```
  int i;
```

```
  int w;
```

```
  int v;
```

```
  int x[MAXN];
```

```
  double ub;
```

```
};
```

// 重量, 下标 0 不用

// 价值, 下标 0 不用

// 存放最大价值, 初始为最小值

// 存放最优解, 全局变量

// 解空间中结点数累计, 全局变量

// 队列中的结点类型

// 结点编号

// 当前结点在搜索空间中的层次

// 当前结点的总重量

// 当前结点的总价值

// 当前结点包含的解向量

// 上界


```
void EnQueue(NodeType e, queue<NodeType> &qu)
```

```
// 结点 e 进队 qu
```

```
{ if (e.i==n)
```

```
// 到达叶子结
```

点

```
{ if (e.v>maxv)
```

```
// 找到更大价值的解
```

```
{ maxv=e.v;
```

```
for (int j=1;j<=n;j++)
```

```
bestx[j]=e.x[j];
```

```
}
```

```
}
```

```
else qu.push(e);
```

```
// 非叶子结点进队
```

```
}
```

→ 在结点进队时判断是否为叶子结点:

- 叶子结点对应一个解
- 叶子结点不再扩展

```
void bfs()
{   int j;
    NodeType e,e1,e2;
    queue<NodeType> qu;

    e.i=0;
    e.w=0; e.v=0;
    e.no=total++;
    for (j=1;j<=n;j++)
        e.x[j]=0;
    bound(e);
    qu.push(e);
```

// 求 0/1 背包的最优解

// 定义 3 个结点
// 定义一个队列

// 根结点置初值, 其层次计为 0

// 求根结点的上界
// 根结点进队

```

while (!qu.empty())                                // 队不空循环
{   e=qu.front(); qu.pop();                          // 出队结点 e

    if (e.w+w[e.i+1]<=W)                            // 剪枝: 检查左孩子结点
    {   e1.no=total++; e1.i=e.i+1;                  // 建立左孩子结

        e1.w=e.w+w[e1.i];
        e1.v=e.v+v[e1.i];
        for (j=1;j<=n;j++)                          // 复制解向量
            e1.x[j]=e.x[j];
        e1.x[e1.i]=1;
        bound(e1);                                    // 求左孩子结点的上界
        EnQueue(e1,qu);                              // 左孩子结点进队操作
    }

    e2.no=total++;                                  // 建立右孩子结点
    e2.i=e.i+1;
    e2.w=e.w; e2.v=e.v;
    for (j=1;j<=n;j++)                              // 复制解向量
        e2.x[j]=e.x[j];
    e2.x[e2.i]=0;
    bound(e2);                                        // 求右孩子结点的上界

    if (e2.ub>maxv)                                  // 若右孩子结点可行, 则进队, 否
        EnQueue(e2,qu);
}

```

点

则被剪枝





结点 $e \rightarrow e1, e2$, 剪枝

- 左孩子 : $e.w + w[e.i+1] \leq W$
- 右孩子 : $e2.ub > maxv$

6.2.2 采用优先队列式分枝限界法求解

采用优先队列式分枝限界法求解就是将一般的队列改为优先队列，但必须设计限界函数，因为优先级是以限界函数值为基础的。

限界函数的设计与前面的相同。这里用大根堆表示活结点表，取优先级为活结点所获得的价值。

```

struct NodeType           // 队列中的结点类型
{   int no;               // 结点编号
    int i;                // 当前结点在搜索空间中的层次
    int w;                // 当前结点的总重量
    int v;                // 当前结点的总价值
    int x[MAXN];          // 当前结点包含的解向量
    double ub;            // 上界

    bool operator<(const NodeType &s) const    // 重载 <
关系函数
    {
        return ub<s.ub;                       //ub 越大越优先出队
    }
};

```

```
void bfs()                                // 求 0/1 背包的最优解
{
    int j;
    NodeType e,e1,e2;                    // 定义 3 个结点
    priority_queue<NodeType> qu;         // 定义一个优先队列（大根堆）
    e.i=0;                               // 根结点置初值，其层次计为 0
    e.w=0; e.v=0;
    e.no=total++;
    for (j=1;j<=n;j++)
        e.x[j]=0;
    bound(e);                             // 求根结点的上界
    qu.push(e);                           // 根结点进队
}
```



```

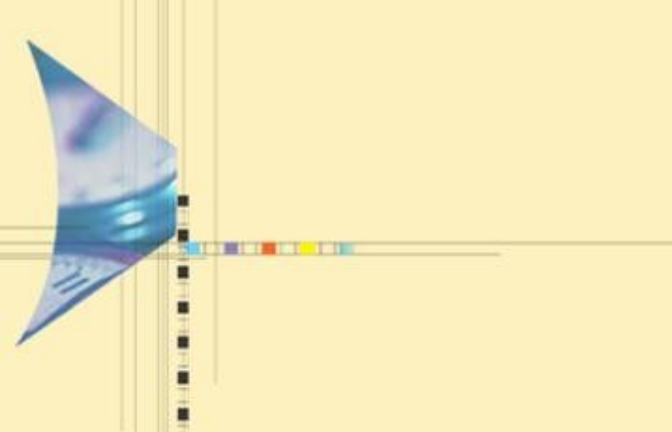
while (!qu.empty())                                // 队不空循环
{
    e=qu.top(); qu.pop();                          // 出队结点 e

    if (e.w+w[e.i+1]<=W)                            // 剪枝: 检查左孩子结点
    {
        e1.no=total++;
        e1.i=e.i+1;                                // 建立左孩子结点
        e1.w=e.w+w[e1.i];
        e1.v=e.v+v[e1.i];
        for (j=1;j<=n;j++) e1.x[j]=e.x[j];        // 复制解向量
        e1.x[e1.i]=1;
        bound(e1);                                // 求左孩子结点的上界
        EnQueue(e1,qu);                            // 左孩子结点进队操作
    }

    e2.no=total++;                                // 建立右孩子结点
    e2.i=e.i+1;
    e2.w=e.w; e2.v=e.v;
    for (j=1;j<=n;j++) e2.x[j]=e.x[j];            // 复制解向量
    e2.x[e2.i]=0;
    bound(e2);                                    // 求右孩子结点的上界
    if (e2.ub>maxv)                                // 若右孩子结点剪枝
        EnQueue(e2,qu);
    }
}

```





【算法分析】无论采用队列式分枝限界法还是优先队列式分枝限界法求解 0/1 背包问题，最坏情况下要搜索整个解空间树，所以最坏时间和空间复杂度均为 $O(2^n)$ ，其中 n 为物品个数。

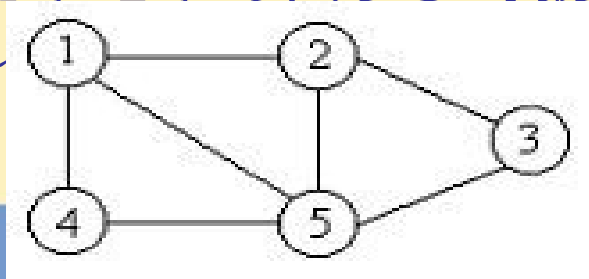


9.3.3 最大团集问题

1. 问题描述

给定无向图 $G=(V, E)$ 。如果 $U \subseteq V$ ，且对任意 $u, v \in U$ 有 $(u, v) \in E$ ，则称 U 是 G 的完全子图。 G 的完全子图 U 是 G 的团当且仅当 U 不包含在 G 的更大的完全子图中。 G 的最大团是指 G 中所含顶点数最多的团。

下图 G 中，子集 $\{1, 2\}$ 是 G 的大小为 2 的完全子图。这个完全子图不是团，因为它被 G 的更大的完全子图 $\{1, 2, 5\}$ 包含。 $\{1, 2, 5\}$ 是 G 的最大团。 $\{1, 4, 5\}$ 和 $\{2, 3, 5\}$ 也是 G 的团。



9.3.3 最大团集问题

2. 上界函数

用变量 **cliqueSize** 表示与该结点相应的团的顶点数；
level 表示结点在子集空间树中所处的层次；用
cliqueSize + n - level + 1 作为顶点数上界 **upperSize** 的值。

在此优先队列式分支限界法中，upperSize 实际上也是优先队列中元素的优先级。算法总是从活结点优先队列中抽取具有最大 **upperSize** 值的元素作为下一个扩展元素。

upperSize 是已找到的团的最大顶点数；

9.3.3 最大团集问题

3. 算法思想

子集树的根结点是初始扩展结点，对于这个特殊的扩展结点，其 `cliqueSize` 的值为 0。

算法在扩展内部结点时，首先考察其左儿子结点。在左儿子结点处，将顶点 i 加入到当前团中，并检查该顶点与当前团中其它顶点之间是否有边相连。当顶点 i 与当前团中所有顶点之间都有边相连，则相应的左儿子结点是可行结点，将它加入到子集树中并插入活结点优先队列，否则就不是可行结点。

接着继续考察当前扩展结点的右儿子结点。当 $\text{upperSize} > \text{bestn}$ 时，右子树中可能含有最优解，此时将右儿子结点加入到子集树中并插入到活结点优先队列中。

9.3.3 最大团集问题

3. 算法思想

算法的 while 循环的终止条件是遇到子集树中的一个叶结点 (即 $n+1$ 层结点) 成为当前扩展结点。

对于子集树中的叶结点, 有 $\text{upperSize} = \text{cliqueSize}$ 。此时活结点优先队列中剩余结点的 upperSize 值均不超过当前扩展结点的 upperSize 值, 从而进一步搜索不可能得到更大的团, 此时算法已找到一个最优解。

- **B**: 界 (已找到团的最大顶点数)。
- **F**: 代价函数值 (估计当前团的最大扩展可能)。

溯;

e
 $F=4$

d
 $F=3$

b
 $F=3$

a
 $B=3$

c
 $B=4$

**c: 极大团 $\{1,3,4,5\}$,
顶点数为 4,
修改界 $B=4$;**

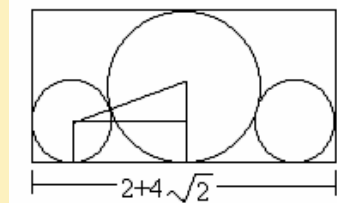
$e: F=4$, 不必搜索.

输出最大团 $\{1,3,4,5\}$, 顶点数为 4.

$$F = \text{cliqueSize} + n - \text{level} + 1$$

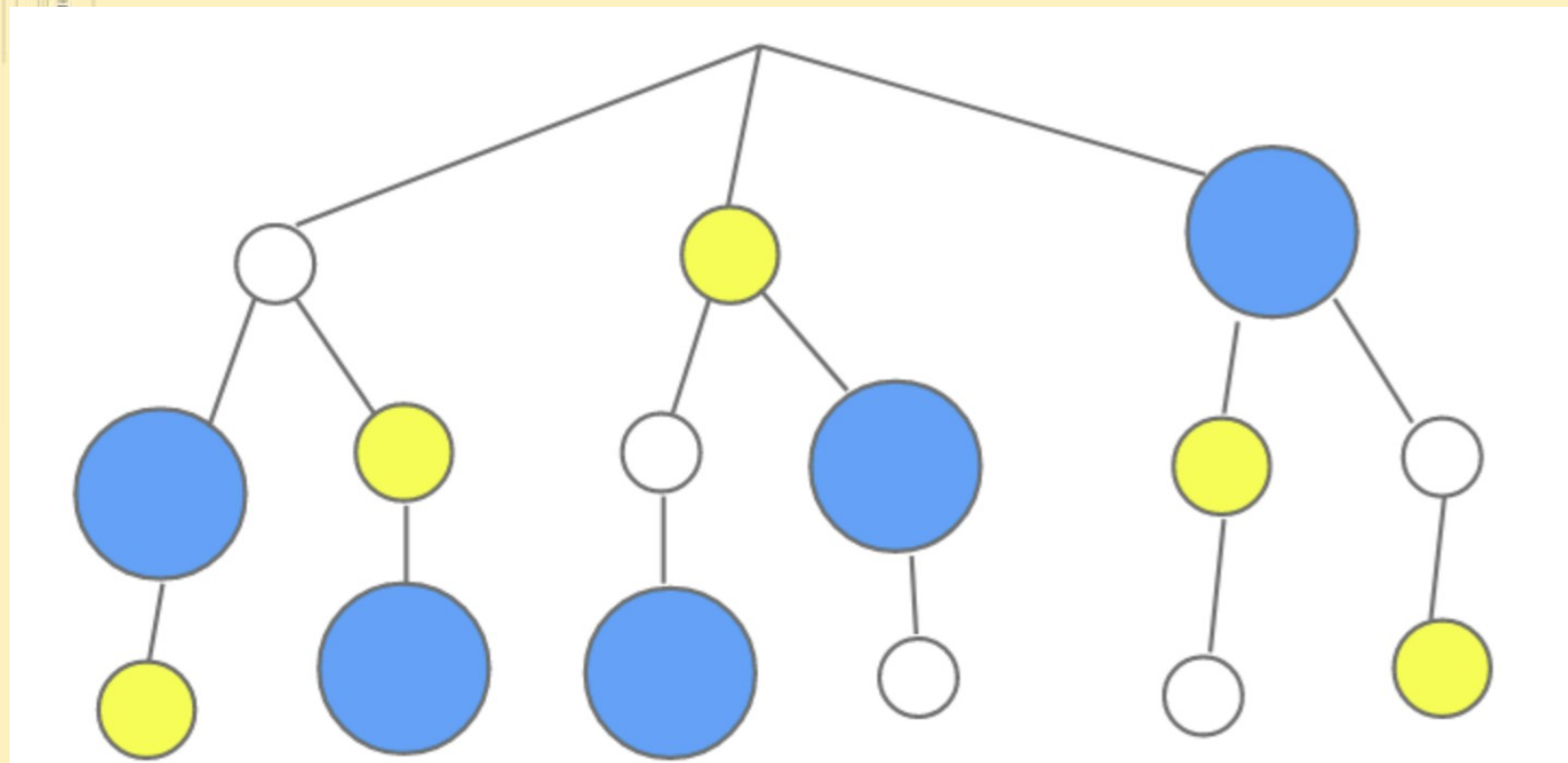
9.3.4 圆排列问题

- 给定 n 个大小不等的圆 $C_1, C_2, C_3, \dots, C_n$ 现要将这 n 个圆排进一个矩形框中，且要求各圆一与矩形框的底边相切。圆排列问题要求从 n 个圆的所有排列中找出有最小长度的圆排列。例如，当 $n=3$ ，且所给的3个圆的半径分别为1, 1, 2时，这3个圆的最小长度的圆排列，其最小长度为 $2+4\sqrt{2}$



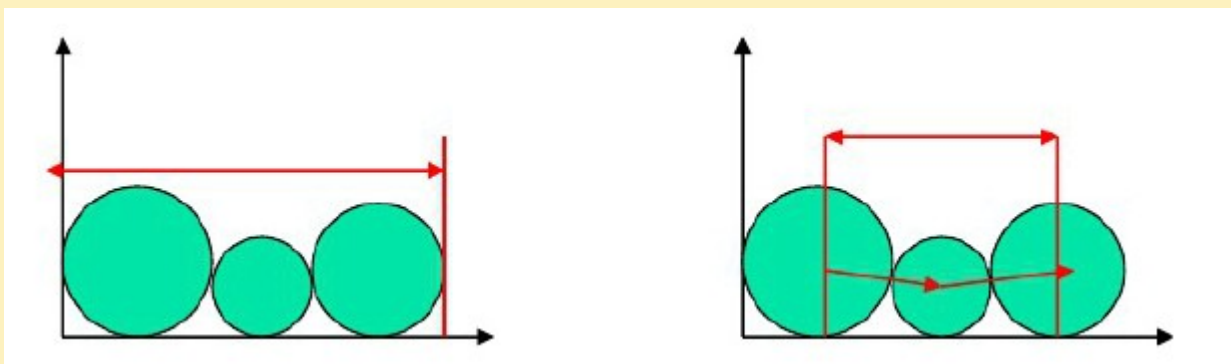
- 对于给定的 n 个圆，设计一个优先队列式分支限界法，计算 n 个圆的最佳排列方案，使其长度达到最小。

9.3.4 圆排列问题



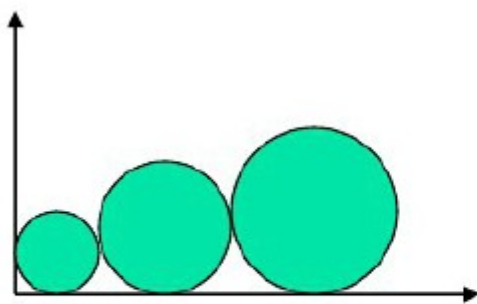
9.3.4 圆排列问题

- 圆排列的解空间是一棵排列树，采用优先队列式分支限界法找出最小圆排列长度。
- 首先，用一个最小堆来表示活节点优先队列，其优先值定义为剩下未排列圆的所有半径和，或者定义为剩下未排列圆的相切距离和。如下图：

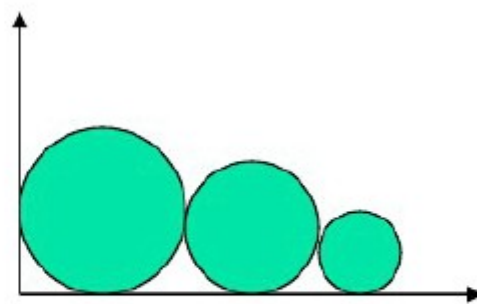


9.3.4 圆排列问题

- 其次，要解决镜像对称问题。例如有 3 个圆，半径分别为 1, 2, 3 与 3, 2, 1 互为镜像排列，具有相同的圆排列长度，因此，只有计算一个就够。这样一来，可以减少一半的计算量。



1, 2, 3



3, 2, 1

9.3.4 圆排列问题

- 解决方法：假设有 n 个圆排列： $1, 2, \dots, n$ 。只要算出 $n!/2$ 种排列，具体排列如下：

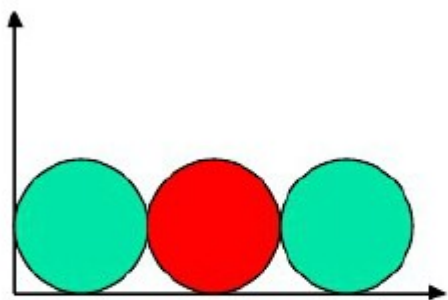
$1, \dots, 2;$	$2, \dots, 3;$	
$1, \dots, 3;$	$2, \dots, 4;$	
$\dots$	$\dots$	$\dots$
$1, \dots, n;$	$2, \dots, n;$	$n-1, \dots, n;$

- 因此只要确定第一个和最后一个位置的数就可以，在排列时，可以从第二个位置到第 $n-1$ 个位置之间排列。

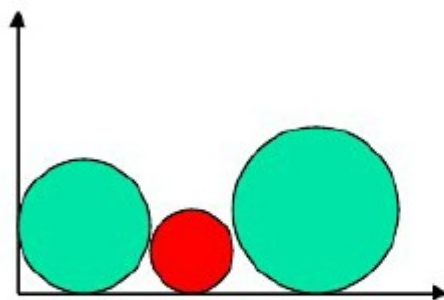


9.3.4 圆排列问题

- 如果所给的 n 个圆中有 k 个圆有相同的半径，则这 k 个圆产生的 $k!$ 个完全相同的圆排列，只计算一个就够，因此，相同半径的圆越多，剪去的节点就越多。
- 解决相同半径圆排列方法如下：如下图有 3 个相同半径的圆，当第一个圆与第二个圆要交换位置时，先判断它们的半径是否相同，若是，则剪枝，否则，



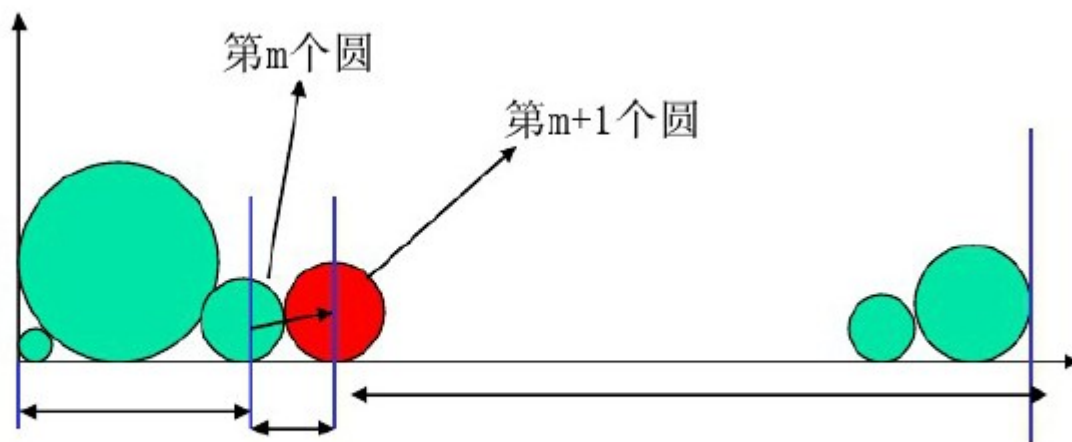
第一个圆与第二个圆半径相同，
交换后总长度也相同。



第一个圆与第二个圆半径不同，
交换后总长度可能不相同。

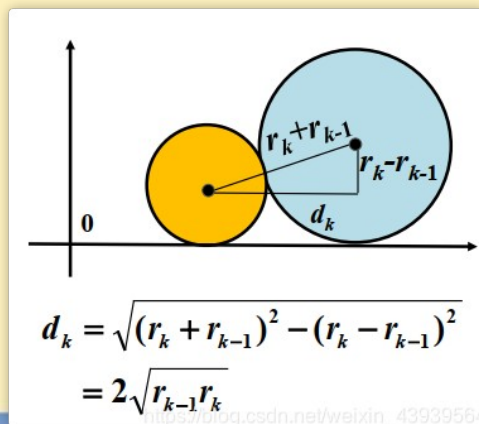
9.3.4 圆排列问题

- 为了加快搜索节点，可以进一步加强下界约束条件。
- 假设，有 n 个圆，其中前面 m 个圆已经确定完位置，还有 $n-m$ 个没确定位置。那么，当搜索到第 $m+1$ 个圆时，
- $\text{min_len}(m) + 2.0 * \text{sqrt}(r[m] * r[m+1]) + \text{len_r}(n-1-m) < \text{min}$



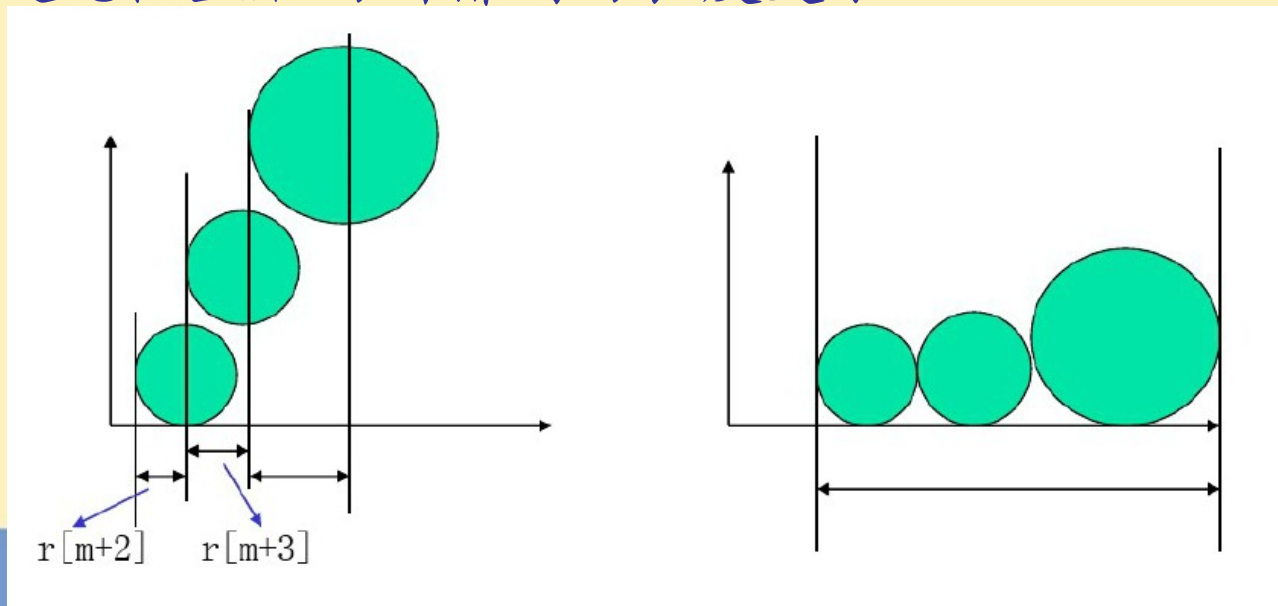
表示前面 m 个圆已排好，长度为 $\text{min_len}(m)$

长度为 $2.0 * \text{sqrt}(r[m] * r[m+1])$



9.3.4 圆排列问题

- 证明:
- 加上 $\text{len_r}(n-1-m)$ 后, $\text{min_len}(m) + 2.0 * \text{sqrt}(r[m] * r[m+1]) + \text{len_r}(n-1-m) < \text{min}$ 总是成立。
- 因为 $\text{len_r}(n-1-m) = r[m+2] + r[m+3] + \dots + r[n]$ 总比确定完位置后的圆排列的长度更小。



9.3.4 圆排列问题

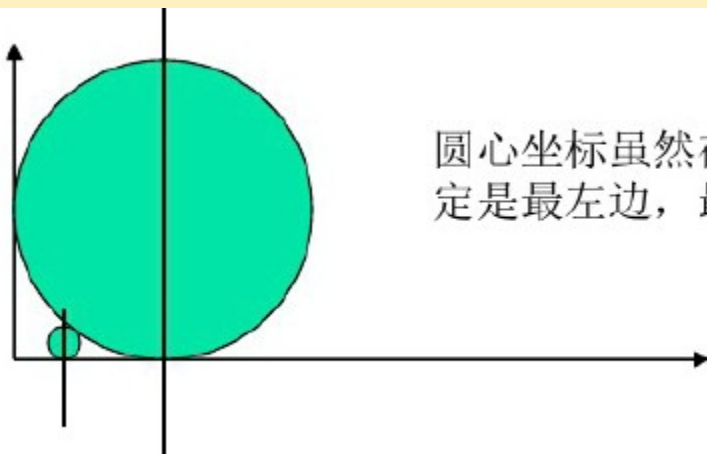
计算当前圆排列的长度。变量 min 记录当前最小圆排列长度。数组 r[] 表示当前圆排列。数组 x[] 则记录当前圆排列中各圆的圆心横坐标。

float Circle::Center(int i) // 计算当前所选择圆的圆心横坐标 (1~i-1 层每一个圆都要计算, 找出一个最大的横坐标, 与之相切, 具体情况可以参考之后的图)

```
{ float temp = 0;
  for(int j=1; j<i; j++) // 注意 i==1 时, 也就是选择第一个圆时, 得到的 x[1] = 0
  { float valuex = x[j] + 2.0*sqrt(r[i]*r[j]);
    if(valuex > temp) temp = valuex;
  }
  return temp;
}
```

9.3.4 圆排列问题

- 计算圆排列的长度：
- `double low=0.0; double high=0.0;`
- `for( j=1;j<=n;j++)`
- `{ if(x[j]-r[j]<low) low=x[j]-r[j]; // 计算最左边的坐标`
- `if(x[j]+r[j]>high) high=x[j]+r[j]; // 计算最右边的坐标`
- `}`
- `if((high-low)<min) min=high-low; // 更新 min`



圆心坐标虽然在最左边，但是该圆的边界不一定是最左边，最右边情况也一样。